CPE415 – ENGINEERING MANAGEMENT

# Software, Project & Program Life Cycles

Systems definition, SDLC methodologies, PMI governance gates, ISO safety standards, and parametric cost estimation

# The Four-Phase Systems Development Cycle

### Phase A: Conception

Initial investigation, technical/economic/operational feasibility, RFP & proposal.

### Phase B: Definition

Front-end loading, requirements traceability matrix, baseline schedule & budget.

### Phase C: Execution

Detail design, PCB/software build, system conversion, postmortem review.

### Phase D: Operation

End-user handover, ongoing maintenance, upgrades, eventual system disposal.

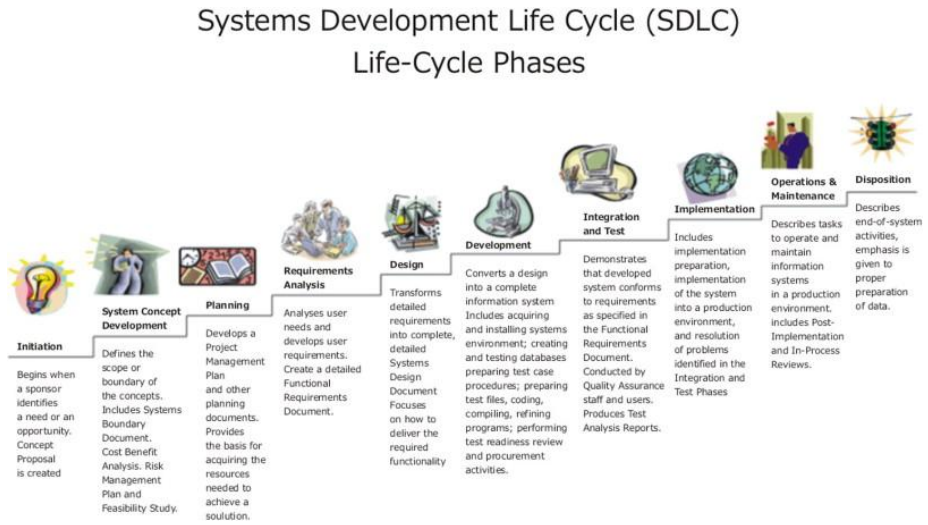



Figure 1: Systems Development Life Cycle (SDC) Progression

# Requirements Traceability & Conversion

## Requirements Traceability Chain

**User Requirement:** Vague operational need ("Call help after shower fall").

**System Requirement:** Functional target ("Detect sudden impact; resist water").

**System Specification:** Measurable criteria ("3-axis MEMS >2.5G; IP68 casing").

**Traceability Matrix:** Maps every technical spec to user needs.

## System Conversion Strategies

**Parallel Installation:** Old and new run together; low risk, high cost.

**Pilot Operation:** Small user cohort rollout; moderate risk and cost.

**Cold Turkey (Big Bang):** Immediate cutover; low cost, extreme failure risk.

**Postmortem Review:** Captures lessons learned before team disbands.

# Program Life Cycle Phases & Six Gates

| Governance Gate | PLC Phase Placement | Core Evaluation Question | Deciding Authority |
|---|---|---|---|
| **1. Authorization** | End of Formulation | Strong strategic business case? | Sponsor & Portfolio Body |
| **2. Plan Approval** | End of Planning | Are 13 subsidiary plans ready? | Program Steering Committee |
| **3. Component Auth** | Prior to Execution | Links directly to target benefit? | Program Governance Board |
| **4. Segment Review** | Periodic in Delivery | Continue, redirect, or terminate? | Governance Participants |
| **5. Transition** | End of Delivery | Named operational owner ready? | Sponsor & Steering Committee |
| **6. Closure Acceptance** | End of Program | Handover & audits complete? | Steering Committee |

# Waterfall Model: Linear & Sequential

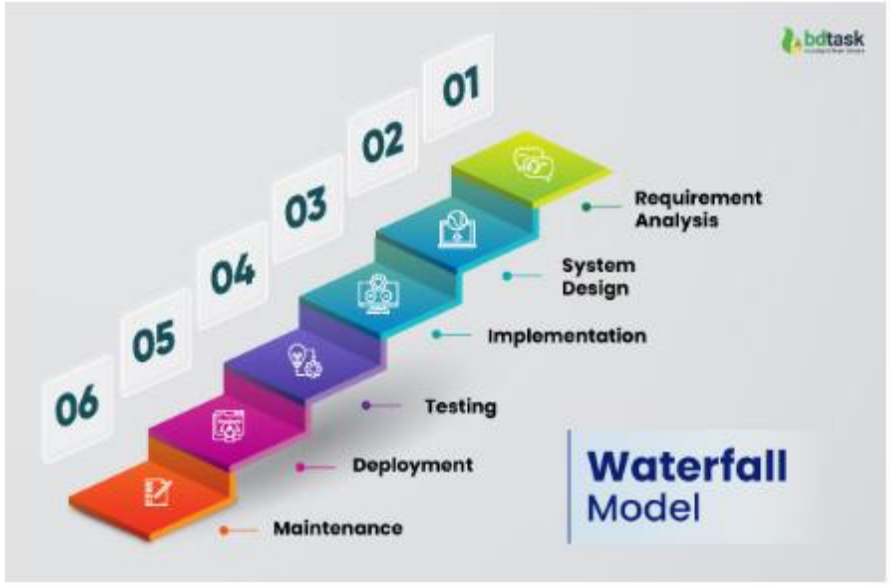



Figure 2: Sequential Waterfall Phase Progression

## Sequential Phase-Gate Architecture

**Overview:** Linear stages where each phase freezes before next starts.

**Proposed By:** Winston W. Royce (1970); Herbert D. Benington (1956).

**Advantages:** Clear milestones, disciplined documentation, simple cost tracking.

**Disadvantages:** Rigid structure; prohibitively expensive late requirement changes.

**Best-Case Scenario:** Fixed, well-understood requirements and stable technology.

**Worst-Case Scenario:** Exploratory R&D or volatile user-facing software.

# V-Model: Safety & Traceability

| | |
|---|---|
| **Overview** | Pairs every design stage with a matching test phase. |
| **Proposed By** | Paul Rook (1986) & German Federal V-Modell. |
| **Advantages** | Strict bidirectional traceability; early defect detection. |
| **Disadvantages** | Inflexible to mid-project changes; heavy documentation. |
| **Best-Case Scenario** | ISO 26262 automotive, avionics, and medical devices. |
| **Worst-Case Scenario** | Consumer startups with rapidly shifting scope. |

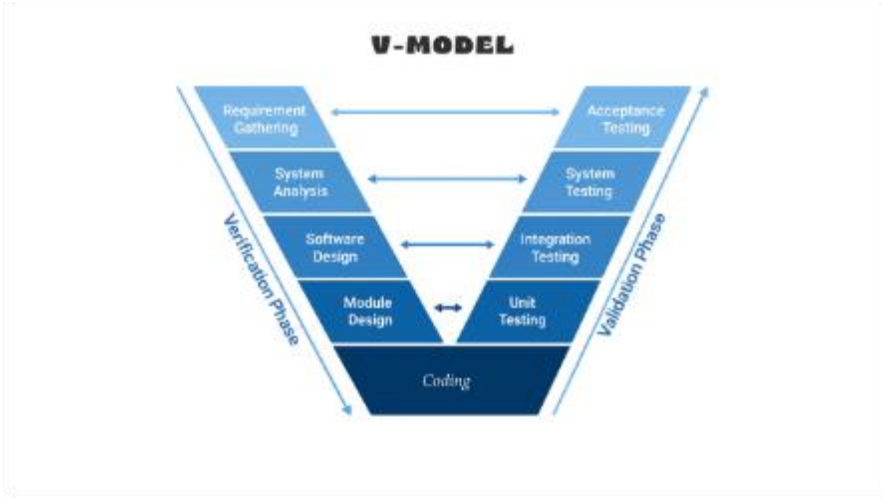



Figure 3: V-Model Design & Verification Symmetry

# Spiral Model: Iterative Risk Analysis

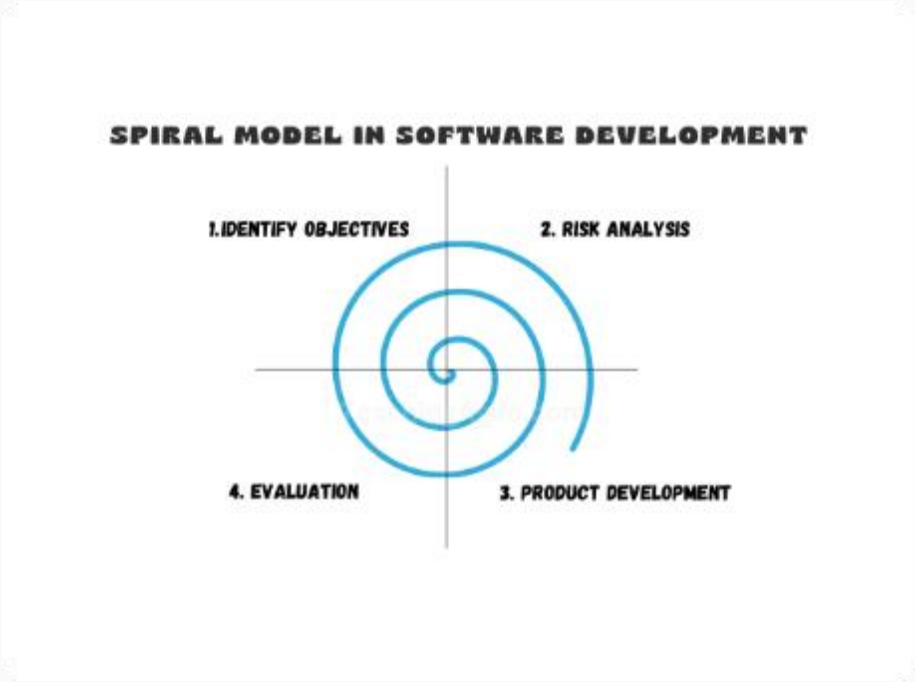



Figure 4: Boehm's Four-Quadrant Risk Spiral

## Risk-First Cyclic Progression

**Overview:** Combines iterative prototyping with systematic Waterfall control.

**Proposed By:** Barry Boehm (1986) in IEEE Computer.

**Advantages:** Continuous risk mitigation; handles evolving technical uncertainty.

**Disadvantages:** Expensive administration; requires specialized risk-analysis expertise.

**Best-Case Scenario:** Large-scale, high-budget aerospace, defense, or enterprise systems.

**Worst-Case Scenario:** Small, low-budget, low-risk projects with simple scope.

# Rapid Prototyping: Early User Validation

| | |
|---|---|
| **Overview** | Builds quick interactive models to solidify user requirements. |
| **Proposed By** | Fred Brooks (1975); Naumann & Jenkins (1982); Martin (1991). |
| **Advantages** | Resolves vague requirements early; prevents costly misbuilds. |
| **Disadvantages** | Users mistake throwaway prototypes for finished production code. |
| **Best-Case Scenario** | UI/UX-heavy apps where users struggle with written specs. |
| **Worst-Case Scenario** | Purely algorithmic backends or shipping unscalable prototype code. |

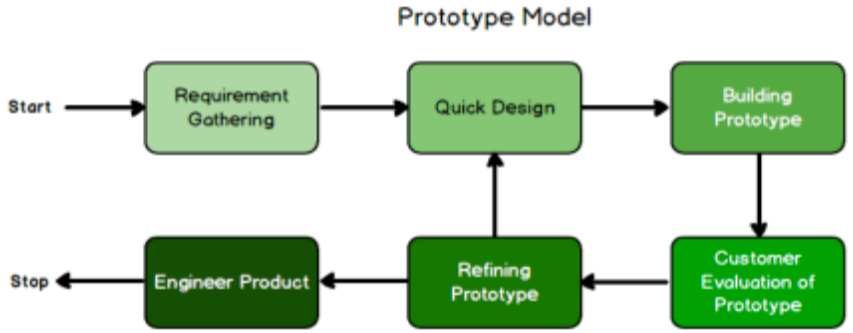



Figure 5: Iterative Prototyping Refinement Cycle

# Agile Scrum: Iterative Sprints

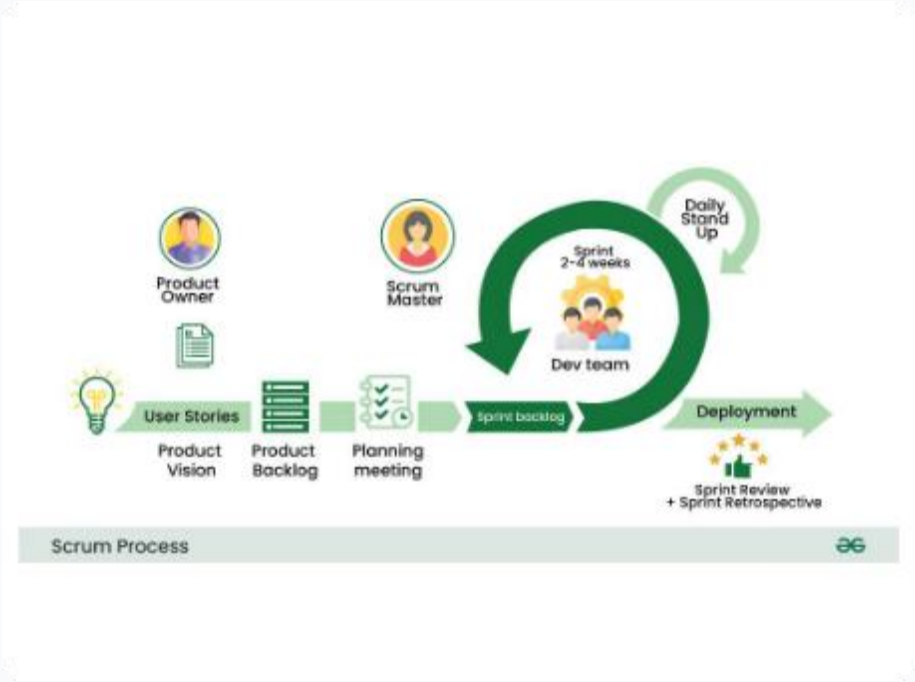



Figure 6: Agile Scrum Backlog, Sprints & Increments

## Incremental Time-Boxed Delivery

**Overview:** Delivers working software increments in 1–4 week sprints.

**Proposed By:** Takeuchi & Nonaka (1986); Schwaber & Sutherland (1995).

**Advantages:** High adaptability; rapid customer feedback; continuous value delivery.

**Disadvantages:** Scope drift risk; harder long-term cost and schedule prediction.

**Best-Case Scenario:** Dynamic commercial SaaS and evolving digital products.

**Worst-Case Scenario:** Fixed-price contracts or standalone physical hardware builds.

ENTERPRISE ARCHITECTURE & HYBRID DELIVERY

# Aligning Organizations with Life Cycles

### Functional Structure

Specialized silos fit linear Waterfall stages.

### Product / Market

Dedicated cross-functional squads power Agile Scrum.

### Matrix Organization

Dual reporting integrates hybrid hardware-software programs.

**Hybrid Project Management:** Synchronizes iterative Agile software sprints with rigid Waterfall hardware manufacturing milestones.

INTERNATIONAL COMPLIANCE STANDARDS

# ISO/IEC/IEEE 12207 & ISO 26262 Safety

### Agreement Processes

Acquisition contract governance across external suppliers.

### Project-Enabling

Portfolio staffing quality infrastructure management.

### Technical Management

Configuration control risk assessment baselines.

### Technical Processes

Traceable architecture coding verification disposal.

### ISO 26262 Embedded Functional Safety

Mandates HARA risk analysis, V-Model traceability, SEooC modules, and ASIL A–D verification rigor.

**ASIL A → D**

PARAMETRIC COST & EFFORT MODELS

# COCOMO II & COSYSMO Estimation

**75%**

### Life Cycle Cost Lock-In

Committed during early concept exploration and architectural definition phases.

### COCOMO II Software Effort (Person-Months)

$$\text{Effort} = A \times (\text{KSLOC})^{E} \times \text{EAF}$$

Modes: Organic (a=2.4, b=1.05), Semidetached (3.0, 1.12), Embedded (3.6, 1.20).

### COSYSMO Equivalent New Requirements

$$\text{ET} = [\,1 - (\text{pM} \times (1 - \text{cM})) - (\text{pR} \times (1 - \text{cR})) - (\text{pD} \times (1 - \text{cD}))\,] \times \text{NT}$$

Drivers: Requirements, Major Interfaces, Operational Scenarios, Critical Algorithms.

# Situational & Contingency Leadership

### S1: Directing

High task, low relationship for junior engineers.

### S2: Coaching

High task, high relationship during early hurdles.

### S3: Supporting

Low task, high relationship builds team confidence.

### S4: Delegating

Autonomous self-managed Agile Scrum squads.

**Vroom Decision Model:** Autocratic (A-1/A-2) for urgent ISO 26262 safety defects vs. Group-Directed (G-2) for rapid prototyping.

OPERATIONAL & STRATEGIC CONTROL

# Control Loops & Strategic Financial Ratios

### 1. Feedforward Control

Pre-execution requirements analysis and COCOMO/COSYSMO budgeting.

### 2. Concurrent Control

Real-time Gantt tracking, daily Scrums, and CI pipelines.

### 3. Feedback Control

Beta testing, postmortem audits, and PLC Closure Acceptance.

| Liquidity (Current / Acid) | Efficiency (Inventory) | Leverage (Debt Ratio) | Profitability (Margin) |
|---|---|---|---|
| Current Assets / Current Liabilities | Cost of Goods Sold / Inventory | Total Debts / Total Assets | Net Profit / Net Sales |

# Module & SDLC References


**Nicholas & Steyn (2021):** *Project Management for Engineering, Business and Technology* (6th ed., Routledge).

**Royce (1970), Rook (1986) & Boehm (1986):** Foundational Waterfall, V-Model, and Spiral SDLC papers (*IEEE*).

**Brooks (1975) & Schwaber & Sutherland (1995):** Rapid Prototyping & Agile Scrum Development Frameworks.

**PMI & ISO Standards:** *Standard for Program Management*; ISO/IEC/IEEE 12207:2026; ISO 26262 Functional Safety.

**Boehm (2000), Farr (2011) & Valerdi (2005):** *COCOMO II*, *COSYSMO* & *Systems Life Cycle Costing* (CRC Press).

# Image Sources



https://ars.els-cdn.com/content/image/3-s2.0-B9781597499613000054-f05-05-9781597499613.jpg

Source: www.Science Direct.com

---



https://www.bdtask.com/blog/assets/plugins/ckfinder/core/connector/php/uploads/images/waterfall%20model(1).jpg

Source: www.bdtask.com

---



https://teachingagile.com/_next/image?url=%2Fblog%2Fenglish%2Fv-model.png&w=3840&q=75

Source: teachingagile.com

---



https://teachingagile.com/blog/english/spiral-model-in-software-development.png

Source: teachingagile.com

---



https://miro.medium.com/1*ZDlW4YN2UVQqD3-_l69dSw.png

Source: medium.com

---



https://media.geeksforgeeks.org/wp-content/uploads/20231213170651/Scrum-Process.jpg

Source: www.geeksforgeeks.org